VHDL Assignment-2

학번: 2024043545
이름: 김예준

Problem 1. Four-input AND / OR Circuit

Requirements

네 개의 입력 a, b, c, d에 대해 AND 연산과 OR 연산을 수행하는 VHDL 회로를 설계한다. Logisim-evolution에서 입력 조합을 변경하여 각 출력이 논리 연산 결과와 일치하는지 확인한다.

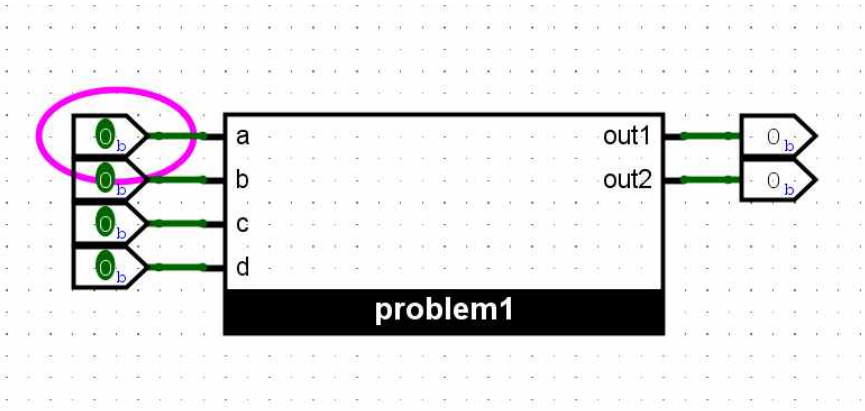
out1은 네 입력의 AND 연산 결과를 출력한다.
out2는 네 입력의 OR 연산 결과를 출력한다.

1. VHDL code

```
library ieee;
use ieee.std_logic_1164.all;
entity problem1 is
    port (
        a, b, c, d : in std_logic;
        out1, out2 : out std_logic
    );
end problem1;
architecture behavioral of problem1 is
begin
    out1 <= a and b and c and d;
    out2 <= a or b or c or d;
end behavioral;
```

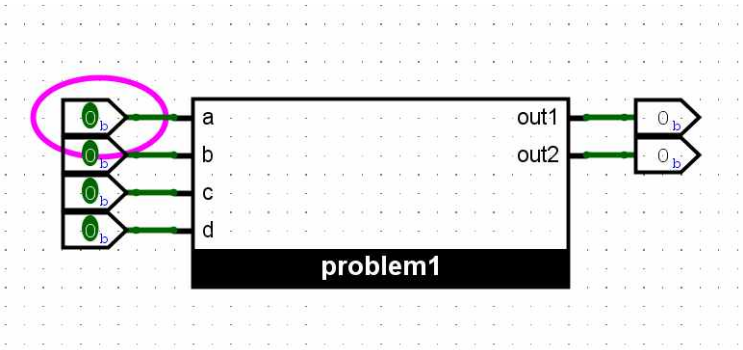
2. Basic circuit

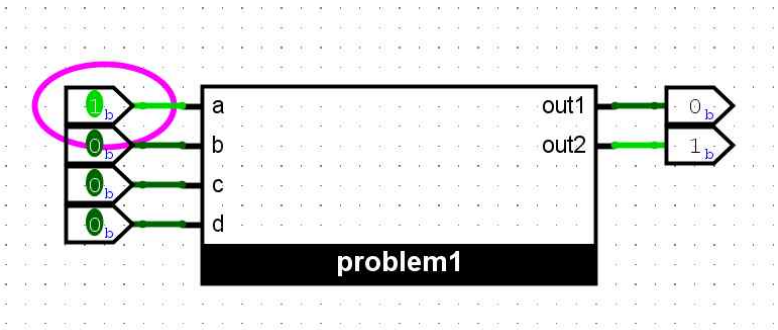
입력 핀 a, b, c, d를 VHDL 블록의 각 입력 포트에 연결하고, 출력 포트 out1과 out2를 각각 출력 핀에 연결한다. 모든 입출력 핀의 비트 수는 1비트로 설정한다.

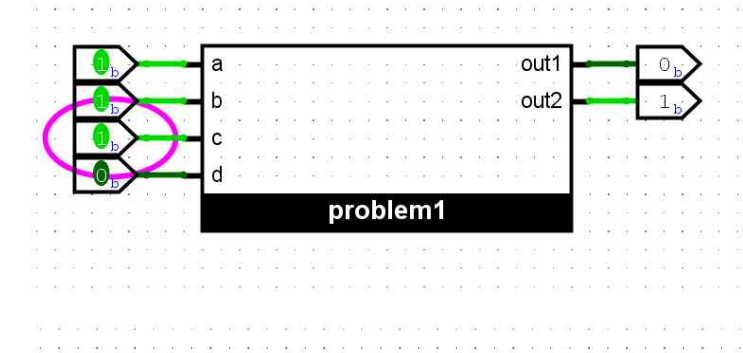


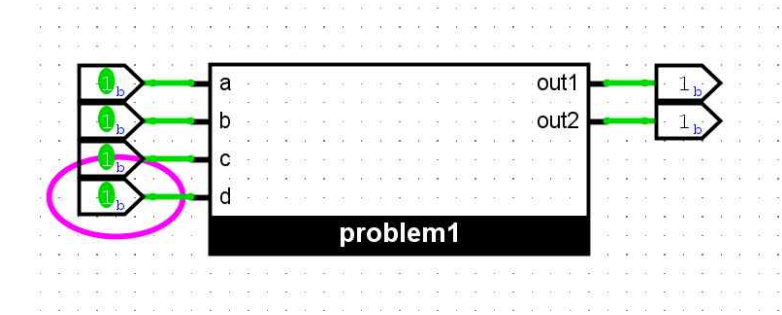
3. Simulation results

출력은 모든 입력이 1일 때만 1이 되며, OR 출력은 하나 이상의 입력이 1일 때 1이 된다.









4. RESULTS

목적

VHDL의 AND와 OR 연산자를 이용하여 4입력 조합논리 회로를 설계하고, 입력 조합에 따른 출력 특성을 확인한다.

회로 구성

네 개의 1비트 입력을 VHDL 블록에 연결하고, 두 개의 출력 핀으로 AND 연산과 OR 연산 결과를 각각 확인하도록 구성하였다.

동작 설명
out1 <= a and b and c and d;는 네 입력이 모두 1일 때만 out1을 1로 설정한다. out2 <= a or b or c or d;는 네 입력 중 하나 이상이 1이면 out2를 1로 설정한다. 두 문장은 동시 신호 할당문이므로 각 출력은 입력 변화에 따라 계산된다.

Problem 2. 8-to-1 Multiplexer

Requirements

8개의 데이터 입력 중 하나를 선택하여 출력하는 8-to-1 멀티플렉서를 VHDL로 설계한다. 3비트 선택 신호에 따라 지정된 데이터 입력이 출력되는지 확인한다.

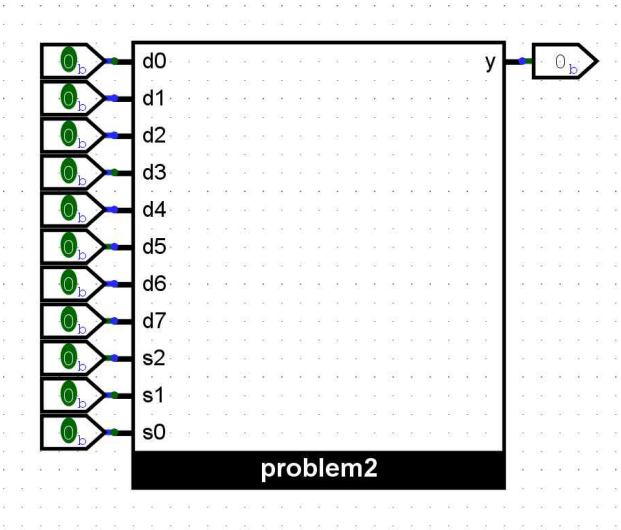
데이터 입력: d0~d7
선택 입력: s2, s1, s0
출력: y
선택 신호는 s2를 최상위 비트, s0를 최하위 비트로 사용한다.

1. VHDL code

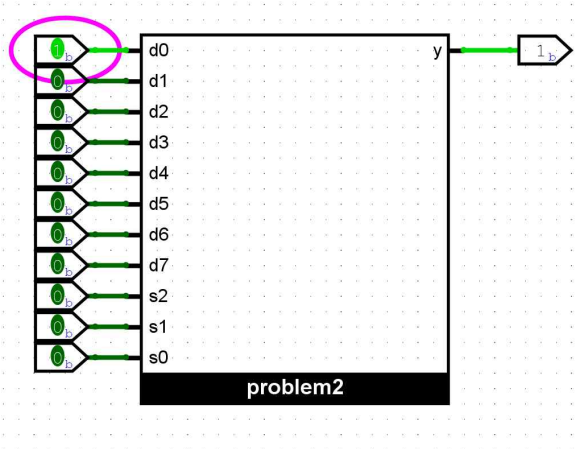
```
library ieee;
use ieee.std_logic_1164.all;
entity problem2 is
    port (
        d0, d1, d2, d3 : in std_logic;
        d4, d5, d6, d7 : in std_logic;
        s2, s1, s0     : in std_logic;
        y              : out std_logic
    );
end problem2;
architecture behavioral of problem2 is
    signal sel : std_logic_vector(2 downto 0);
begin
    sel <= s2 & s1 & s0;
    with sel select
        y <= d0 when "000",
             d1 when "001",
             d2 when "010",
             d3 when "011",
             d4 when "100",
             d5 when "101",
             d6 when "110",
             d7 when "111",
             '0' when others;
end behavioral;
```

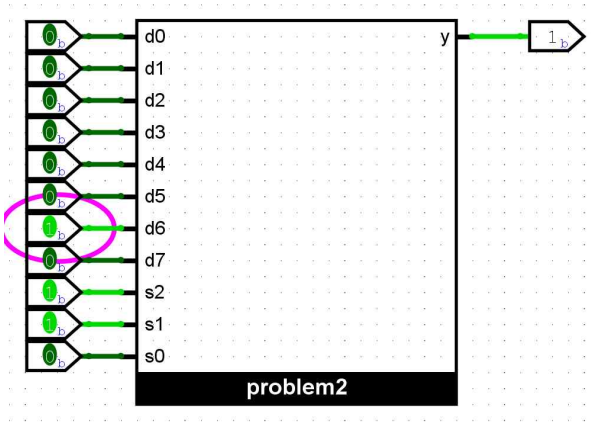
2. Basic circuit

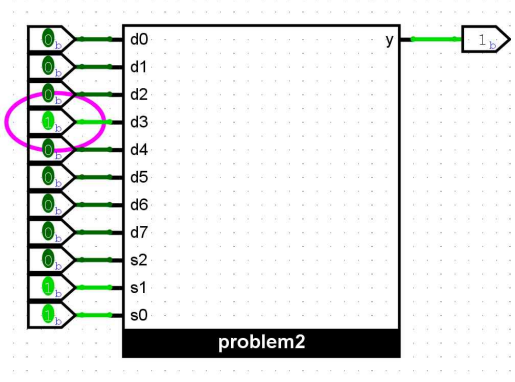
데이터 입력용 핀 8개와 선택 입력용 핀 3개를 VHDL 블록에 연결한다. 출력 y에는 1비트 출력 핀을 연결한다. 모든 핀은 1비트로 설정한다.

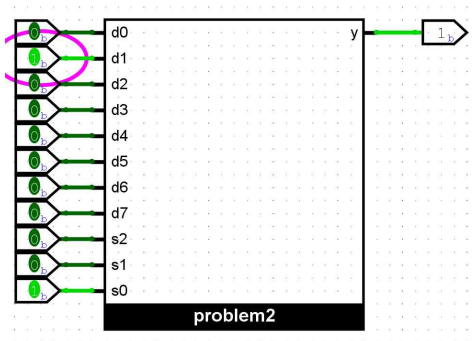


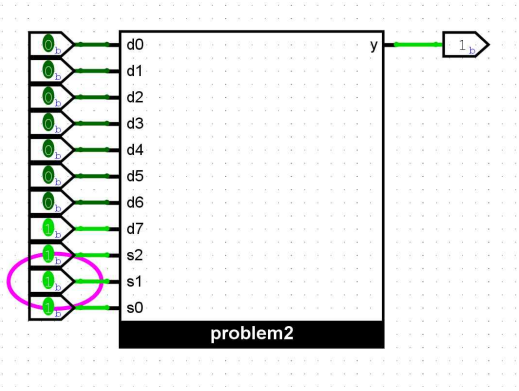
3. Simulation results

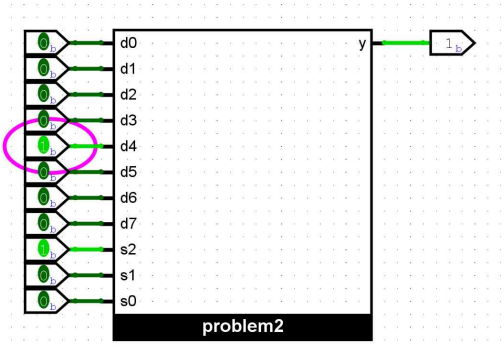


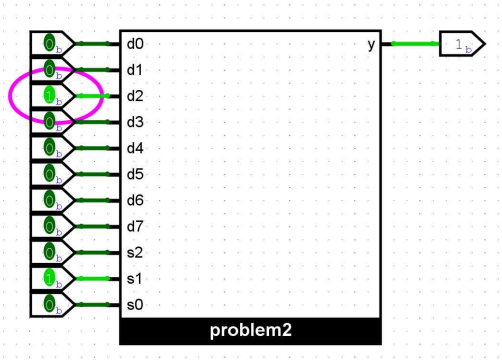


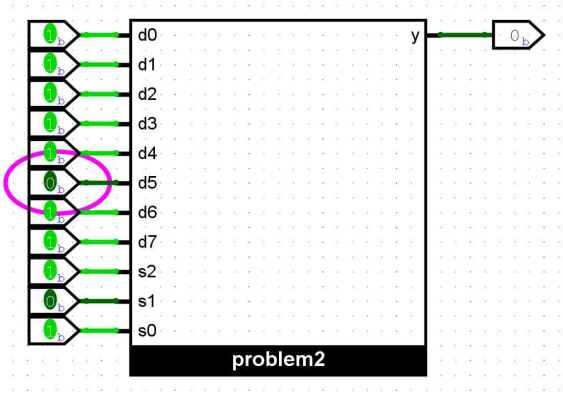


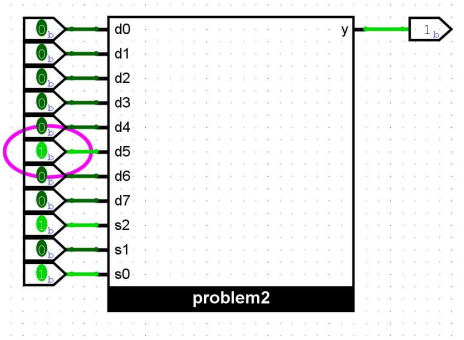












4. RESULTS

목적

3비트 선택 신호를 이용하여 8개의 데이터 입력 중 하나를 출력으로 전달하는 멀티플렉서의 동작을 확인한다.

회로 구성

8개의 데이터 입력과 3개의 선택 입력을 VHDL 블록에 연결하고, 출력 y를 출력 핀에 연결하였다.

동작 설명

연결 연산자 &를 이용하여 s2, s1, s0를 3비트 신호 sel로 구성하였다. with sel select 문은 sel 값에 따라 d0부터 d7까지의 입력 중 하나를 y에 할당한다. 예를 들어 sel이 101이면 d5가 선택되므로 y는 d5와 같은 값을 가져야 한다. 검증에서는 선택된 입력이 출력에 전달되는지와 선택되지 않은 입력의 변화가 출력에 영향을 주지 않는지를 함께 확인한다.

Problem 3. Letter Display Using a 7-Segment Display

Requirements

VHDL로 7-segment 표시 회로를 설계하여 문자 a, b, c, d, f, g를 표시한다. 3비트 선택 신호를 사용하여 표시할 문자를 지정한다.

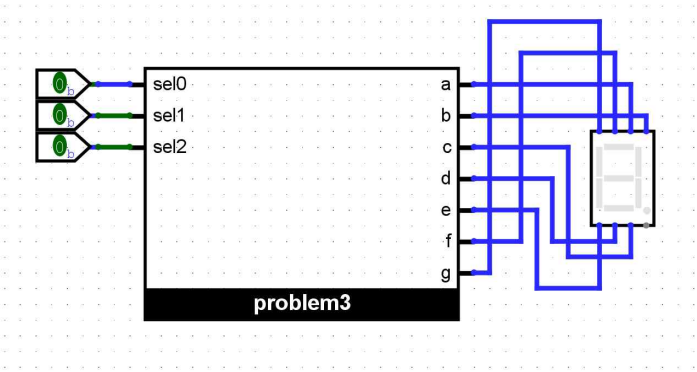
7-segment의 표현 형태를 고려하여 문자는 a, b, C, d, F, g의 모양으로 표시한다. 각 세그먼트는 출력이 1일 때 켜지도록 구성한다.

1. VHDL code

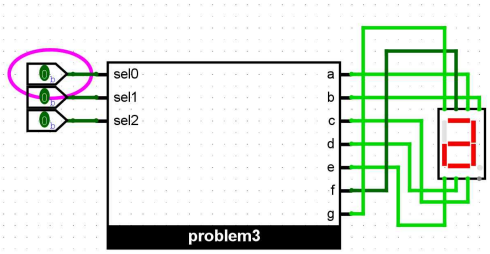
```
library ieee;
use ieee.std_logic_1164.all;
entity problem3 is
    port (
        sel0, sel1, sel2      : in std_logic;
        a, b, c, d, e, f, g : out std_logic
    );
end problem3;
architecture behavioral of problem3 is
    signal sel : std_logic_vector(2 downto 0);
    signal seg : std_logic_vector(6 downto 0);
begin
    sel <= sel2 & sel1 & sel0;
    -- Segment order: a b c d e f g
    with sel select
        seg <= "1111101" when "000", -- a
               "0011111" when "001", -- b
               "1001110" when "010", -- C
               "0111101" when "011", -- d
               "1000111" when "100", -- F
               "1111011" when "101", -- g
               "0000000" when others;
    a <= seg(6);
    b <= seg(5);
    c <= seg(4);
    d <= seg(3);
    e <= seg(2);
    f <= seg(1);
    g <= seg(0);
end behavioral;
```

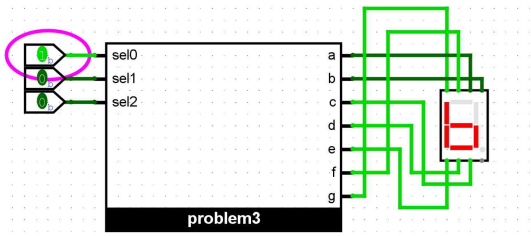
2. Basic circuit

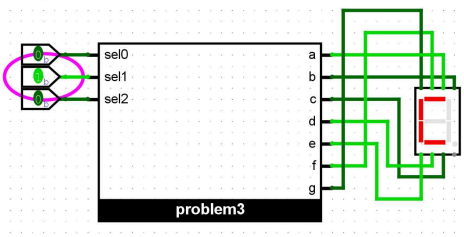
선택 입력 sel0, sel1, sel2에 각각 1비트 입력 핀을 연결한다. VHDL 블록의 a~g 출력을 7-segment 표시기의 해당 세그먼트에 연결한다. 소수점 dp는 사용하지 않으므로 0에 연결한다.

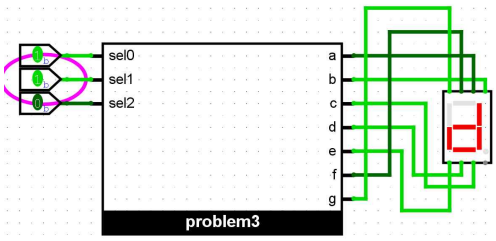


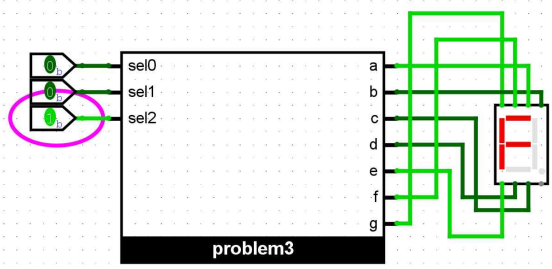
3. Simulation results

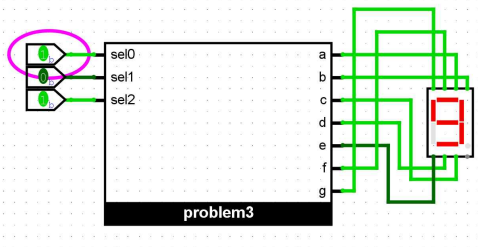


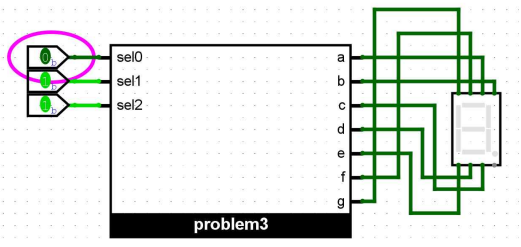












4. RESULTS

목적
문자별 세그먼트 점등 패턴을 VHDL로 구현하고, 선택 신호에 따라 지정된 문자가 표시되는지 확인한다.

회로 구성
3개의 입력 핀을 선택 입력에 연결하고, 7개의 출력을 표시기의 각 세그먼트에 연결하였다. 각 출력은 1일 때 해당 세그먼트를 켠다.

동작 설명
선택 입력을 3비트 벡터 sel로 구성하고, 선택값에 해당하는 7비트 패턴을 seg에 할당하였다. seg의 각 비트는 a~g 출력과 연결된다. 예를 들어 문자 F의 패턴은 1000111이므로 a, e, f, g 세그먼트가 켜진다. 지정하지 않은 선택값 110과 111에서는 모든 세그먼트를 끄도록 하였다.

Problem 4. 25-Person Voting Machine

Requirements

25명의 투표 결과를 입력받아 찬성표가 과반수인 경우 1을 출력하는 VHDL 회로를 설계한다.

입력 vote는 25비트 벡터이다.
각 비트의 1은 찬성, 0은 반대를 의미한다.
찬성표가 13개 이상이면 result=1을 출력한다.
찬성표가 12개 이하이면 result=0을 출력한다.

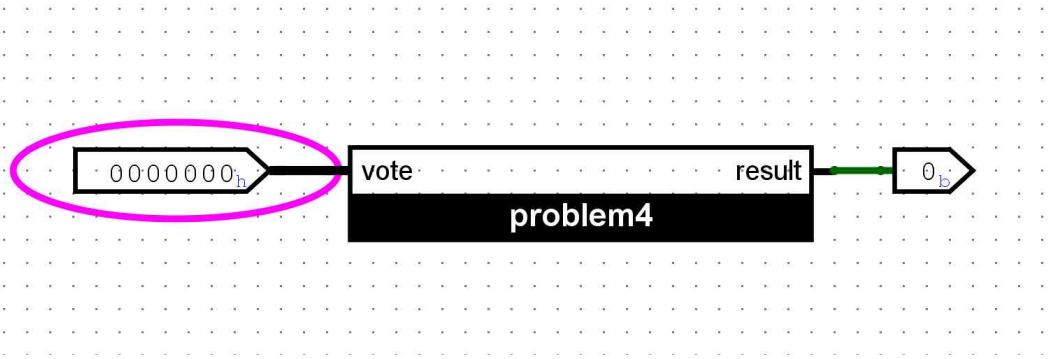
1. VHDL code

```
library ieee;
use ieee.std_logic_1164.all;
entity problem4 is
    port (
        vote   : in std_logic_vector(24 downto 0);
        result : out std_logic
    );
end problem4;
architecture behavioral of problem4 is
begin
    process(vote)
        variable count : integer range 0 to 25;
    begin
        count := 0;
        for i in vote'range loop
            if vote(i) = '1' then
                count := count + 1;
            end if;
```

```
        end loop;
        if count >= 13 then
            result <= '1';
        else
            result <= '0';
        end if;
    end process;
end behavioral;
```

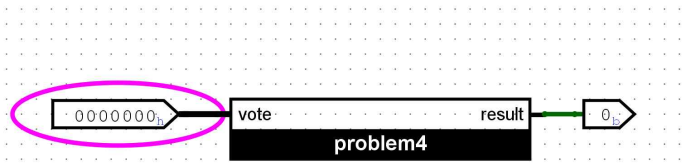
2. Basic circuit

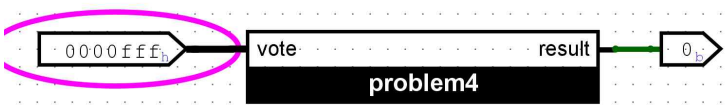
25비트 입력 신호를 VHDL 블록의 vote 포트에 연결하고, result를 출력 핀 또는 LED에 연결한다. 입력 비트의 1 개수를 변경하여 과반수 여부를 확인한다.

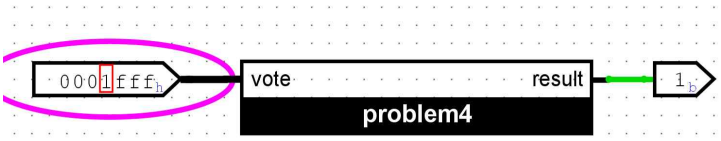


3. Simulation results

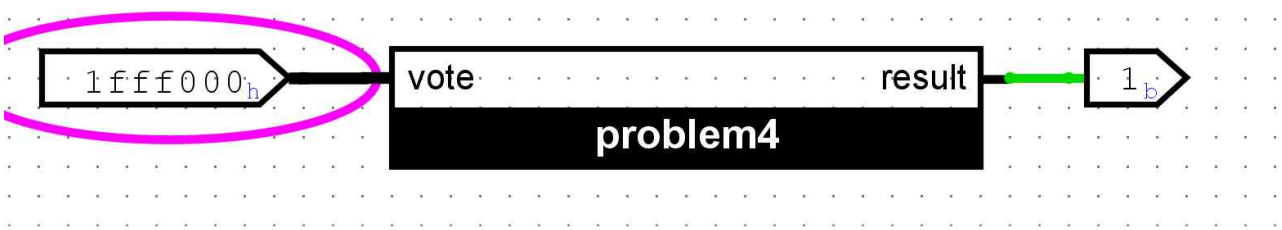
25명의 과반수는 13명이므로, 12표와 13표를 기준으로 출력이 바뀌는지 확확인한다.











4. RESULTS

목적
다수의 입력에서 1의 개수를 계산하고, 과반수 기준에 따라 출력을 결정하는 투표 회로를 구현한다.

회로 구성
25비트 vote 입력과 1비트 result 출력으로 구성하였다. vote의 각 비트는 한 명의 투표 결과를 나타낸다.

동작 설명
vote가 변경될 때 process가 실행된다. 먼저 count를 0으로 초기화한 후, 반복문으로 25개 비트를 검사한다. 값이 1인 비트를 발견할 때마다 count를 1씩 증가시키고, 최종 개수가 13 이상이면 result에 1을 할당한다.

count는 process가 실행될 때마다 초기화하므로 이전 입력에서 계산한 값이 누적되지 않는다. 이 회로는 시간에 따라 투표를 누적하는 순차회로가 아니라, 현재 입력의 찬성표 수를 판단하는 조합논리 회로이다.

Problem 5. Counting Ones and Displaying the Result

Requirements

13비트 입력 vt에서 1의 개수를 계산하고, 그 결과를 7-segment 표시기에 출력하는 VHDL 모듈을 설계한다.

1의 개수가 0~9이면 해당 숫자를 표시한다.
1의 개수가 10~15이면 A, B, C, D, E, F에 해당하는 형태를 표시한다.
en=0이면 모든 세그먼트를 끈다.
en=1이면 1의 개수에 해당하는 값을 표시한다.

7-segment에서는 B와 D를 각각 소문자 형태인 b와 d로 표시한다.

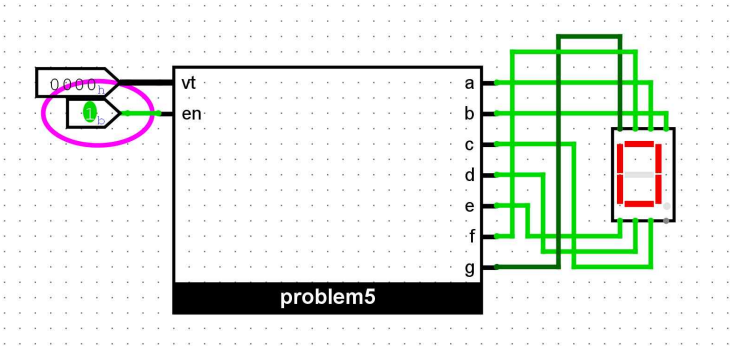
1. VHDL code

```
library ieee;
use ieee.std_logic_1164.all;
entity problem5 is
    port (
        vt                    : in std_logic_vector(12 downto 0);
        en                    : in std_logic;
        a, b, c, d, e, f, g : out std_logic
    );
end problem5;
architecture behavioral of problem5 is
    signal seg : std_logic_vector(6 downto 0);
begin
```

```
    process(vt, en)
        variable count : integer range 0 to 15;
    begin
        count := 0;
        for i in vt'range loop
            if vt(i) = '1' then
                count := count + 1;
            end if;
        end loop;
        if en = '1' then
            -- Segment order: a b c d e f g
            case count is
                when 0  => seg <= "1111110";
                when 1  => seg <= "0110000";
                when 2  => seg <= "1101101";
                when 3  => seg <= "1111001";
                when 4  => seg <= "0110011";
                when 5  => seg <= "1011011";
                when 6  => seg <= "1011111";
                when 7  => seg <= "1110000";
                when 8  => seg <= "1111111";
                when 9  => seg <= "1111011";
                when 10 => seg <= "1110111"; -- A
                when 11 => seg <= "0011111"; -- b
                when 12 => seg <= "1001110"; -- C
                when 13 => seg <= "0111101"; -- d
                when 14 => seg <= "1001111"; -- E
                when 15 => seg <= "1000111"; -- F
                when others => seg <= "0000000";
            end case;
        else
            seg <= "0000000";
        end if;
    end process;
    a <= seg(6);
    b <= seg(5);
    c <= seg(4);
    d <= seg(3);
    e <= seg(2);
    f <= seg(1);
    g <= seg(0);
end behavioral;
```

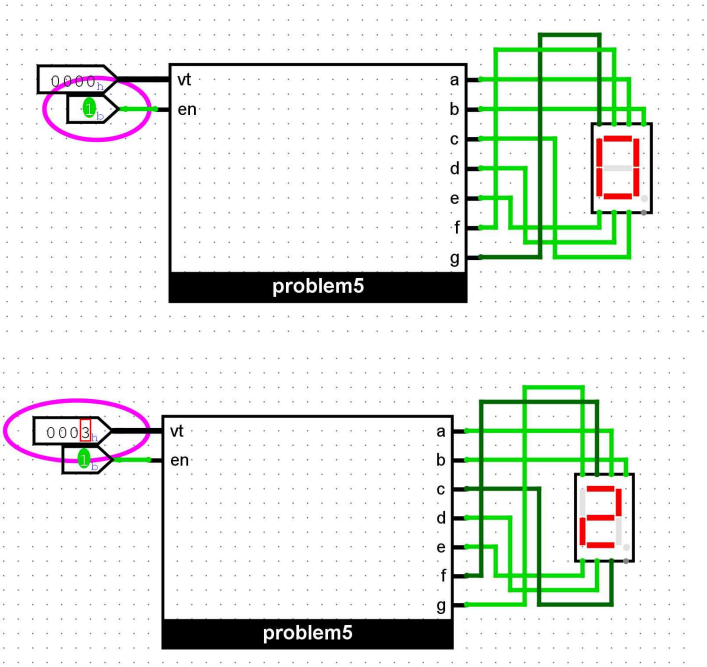
2. Basic circuit

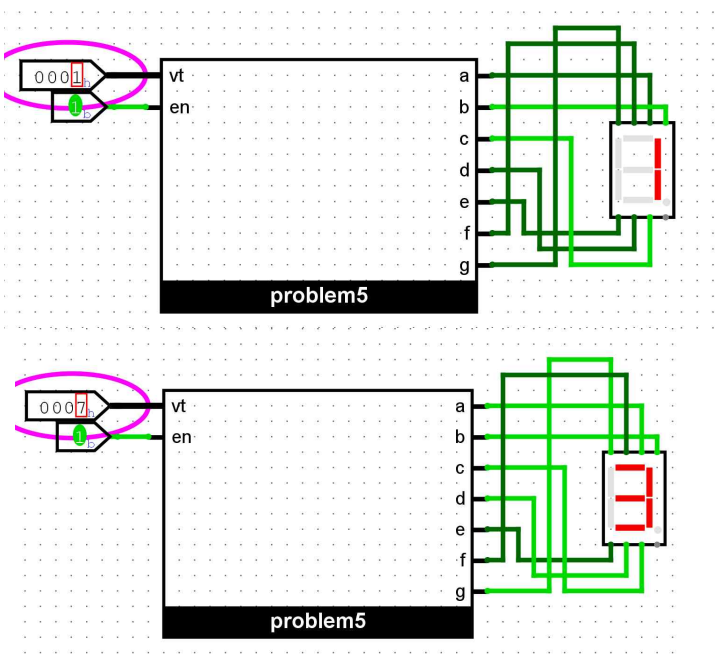
13비트 입력 신호를 vt에 연결하고, 1비트 입력 핀을 en에 연결한다. 출력 a~g는 7-segment 표시기의 해당 세그먼트에 연결한다. 소수점 dp는 0에 연결한다.

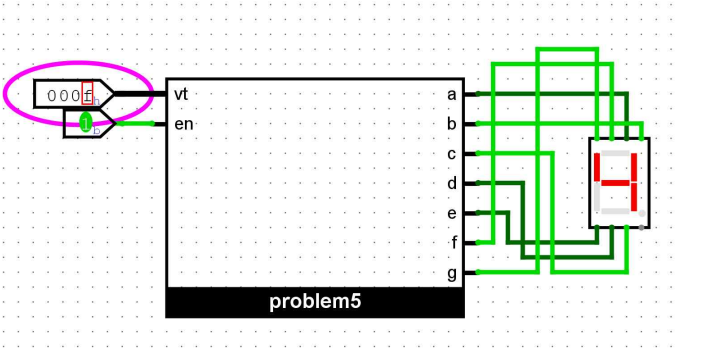


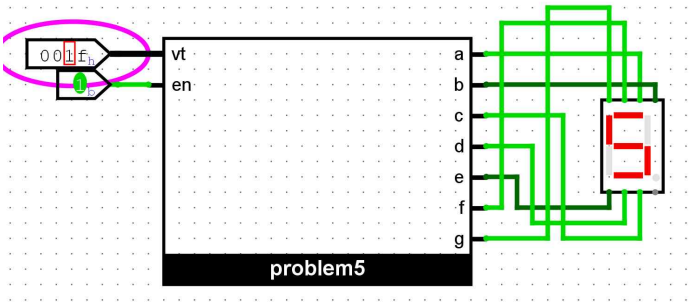
3. Simulation results

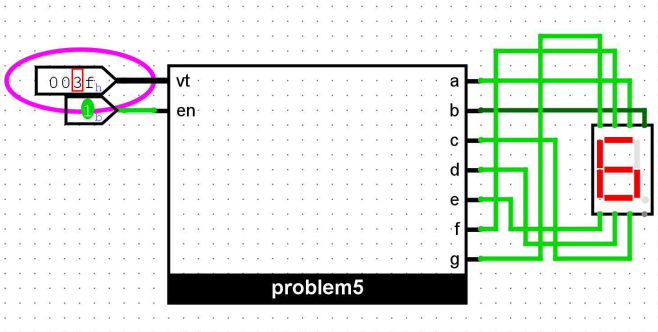
en=1로 설정한 상태에서 vt의 1 개수를 0부터 13까지 변경한다.

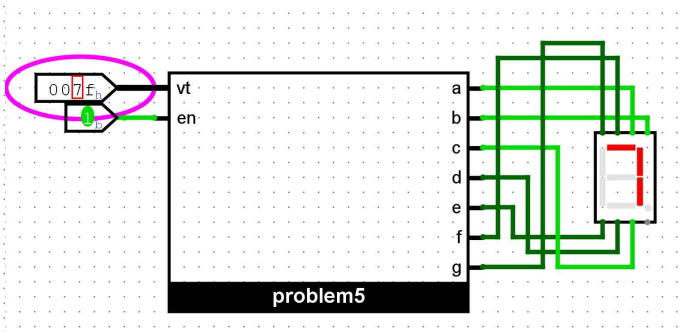


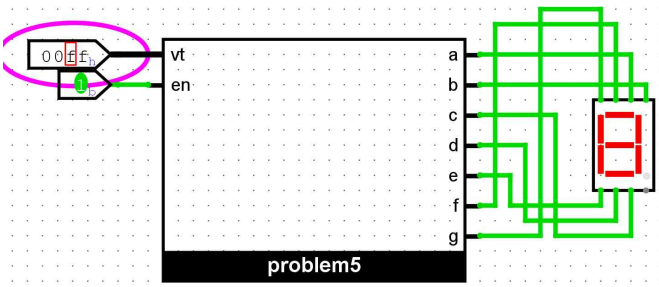


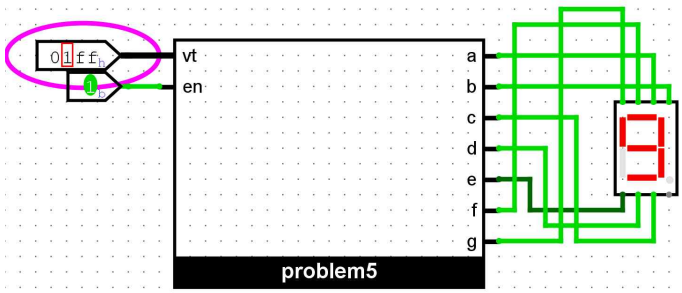


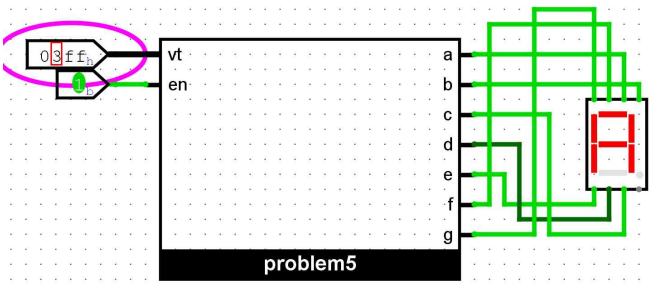


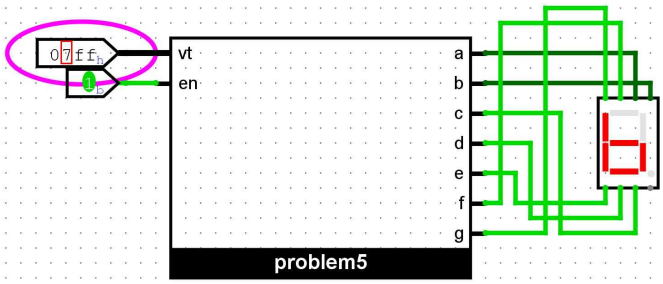

003f
0
vt
en
a
b
c
d
e
f
g
problem5

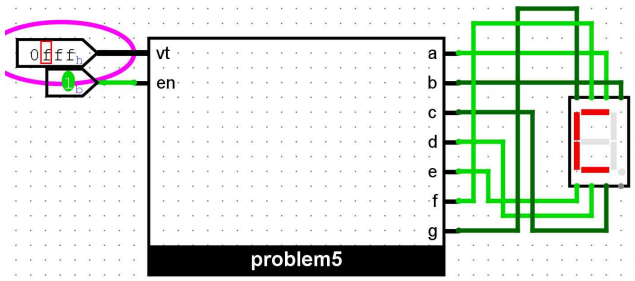

007f
1
vt
en
a
b
c
d
e
f
g
problem5

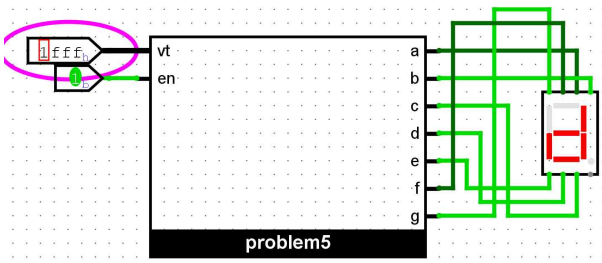

00ff
1
vt
en
a
b
c
d
e
f
g
problem5

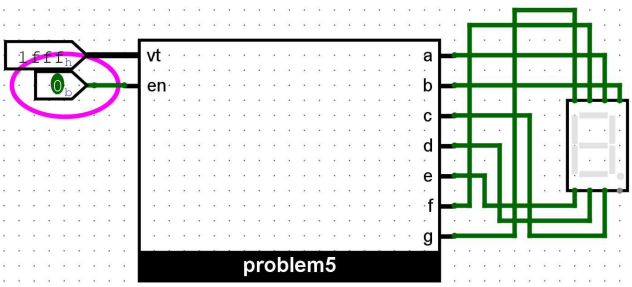

01ff
1
vt
en
a
b
c
d
e
f
g
problem5


03ff
1
vt
en
a
b
c
d
e
f
g
problem5


07ff
0
vt
en
a
b
c
d
e
f
g
problem5


0fff
1
vt
en
a
b
c
d
e
f
g
problem5


1fff
1
vt
en
a
b
c
d
e
f
g
problem5


1fff
0
vt
en
a
b
c
d
e
f
g
problem5

4. RESULTS

목적
입력 벡터에서 1의 개수를 계산하는 기능과 7-segment 표시 기능을 결합하고, enable 신호로 출력 표시를 제어한다.

회로 구성
13비트 입력 vt, 1비트 입력 en, 7개의 세그먼트 출력으로 구성하였다. vt는 개수 계산에 사용하고, en은 표시 여부를 결정한다.

동작 설명
vt 또는 en이 변경되면 process가 실행된다. count를 0으로 초기화한 뒤 반복문으로 vt의 각 비트를 확인하여 1의 개수를 계산한다.

en=1이면 case 문을 이용하여 count에 해당하는 세그먼트 패턴을 출력한다. en=0이면 모든 세그먼트에 0을 할당하여 표시기를 끈다. 따라서 숫자 0의 표시와 표시기 꺼짐은 서로 다른 출력 상태이다.

또한 입력값이 감소할 때도 count를 처음부터 다시 계산하므로, 이전 입력의 개수가 결과에 누적되지 않는다.

표시 범위에 대한 검토
과제에는 1의 개수가 10~15일 때 A~F를 표시하도록 제시되어 있다. 이에 따라 코드에는 14와 15에 대한 E, F 패턴도 포함하였다.

다만 vt는 13비트이므로 1의 개수는 최대 13이다. 따라서 지정된 입력 조건에서 실제로 확인할 수 있는 표시는 0~9와 A, b, C, d이며, E와 F는 vt 입력만으로 도달할 수 없다.

Overall Discussion

본 과제의 다섯 회로는 모두 현재 입력에 따라 출력이 결정되는 조합논리 회로로 설계하였다. AND·OR 회로는 기본 논리 연산을 수행하고, 멀티플렉서는 선택 신호에 따라 입력을 전달한다. 문자 표시 회로는 선택값을 세그먼트 패턴으로 변환하며, 투표 회로와 개수 표시 회로는 입력 벡터에서 1의 개수를 계산한다.

검증에서는 일반적인 입력뿐 아니라 출력이 바뀌는 경계 조건도 확인해야 한다. 투표 회로에서는 찬성표 12개와 13개가 경계 조건이며, 개수 표시 회로에서는 9에서 A로 바뀌는 조건과 en에 따른 표시 전환이 주요 확인 항목이다.

VHDL 코드와 Logisim 회로를 함께 검토할 때는 포트 이름, 비트 수, 선택 신호의 순서, 세그먼트 연결 위치를 일치시켜야 한다. 특히 7-segment의 출력 패턴 순서를 a~g로 통일하여 코드와 실제 배선의 대응 관계를 확인하도록 구성하였다.